

RSA and Digital Signatures

Import

The **RSA** cryptographic algorithm is a public-key cryptosystem. It is an asymmetric cryptographic algorithm where each user has two keys, one private and one public. It uses elements from number theory and in combination with particularly **large keys** achieves arithmetic remainder encryption that makes decryption by factorization impossible. It is considered to be a secure cryptographic algorithm to this day.

General Framework

The RSA algorithm involves calculations on large numbers. These calculations cannot be carried out directly on the machine using simple arithmetic operators in programs, because these operators can only operate on classic data types, such as 32-bit and 64-bit integers. The numbers involved in RSA algorithms are typically larger than 512 bits. For example, to multiply two 32-bit integers **a** and **b**, we simply need to use **a*b** in our program. However, if the numbers are large, this cannot be done. Instead, we need to use an algorithm (i.e. a

function) to calculate their products, thus allowing us to overcome the limits of the machine and specifically of the circuits of the arithmetic and logic unit

her.

There are several libraries that can perform arithmetic operations on arbitrary-sized integers. In this lab, we will use the **bn** (multiprecision integer arithmetics) large number library provided by **openssl1**. H

This library provides the **BIGNUM2 type**, which can represent any large number. number. It also provides an API that implements various functions, such as addition, multiplication, exponentiation, remainder arithmetic, etc. In this exercise, all the material from the theoretical part of the RSA course uploaded to eclass will be used.

¹ <https://www.openssl.org>

² <https://www.openssl.org/docs/man1.0.2/man3/bn.html>

BIGNUM APIs

All the APIs for large numbers provided by the bn library are available at <https://www.openssl.org/docs/man1.0.2/man3/bn.html>. In the continuation of the exercise, Some of the functions needed for this lab exercise are described.

- Some library functions require **temporary** variables. Since dynamically allocating memory to create **BIGNUMs** is quite expensive, when used in conjunction with repeated calls to the subroutine, a **BN_CTX** structure is created to hold temporary BIGNUM variables used by library functions. We need to create a such a structure and pass it (pass it as an argument) to the functions that require it.

```
BN_CTX *ctx = BN_CTX_new()
```

- **Initializing** a BIGNUM variable:

```
BIGNUM *a = BN_new()
```

- **Assigning** a value to a BIGNUM variable: There are many ways to assign a value in a BIGNUM variable.

```
// Assign a value from a decimal number string
BN_dec2bn(&a, "12345678901112231223");
// Assign a value from a hex number string
BN_hex2bn(&a, "2A3B4C55FF77889AED3F");
// Generate a random number of 128 bits
BN_rand(a, 128, 0, 0);
// Generate a random prime number of 128 bits
BN_generate_prime_ex(a, 128, 1, NULL, NULL, NULL);
```

- **Printing** a large number:

```
void printBN(char *msg, BIGNUM * a)
{
    // Convert the BIGNUM to number string
```

```

char * number_str = BN_bn2dec(a);
// Print out the number string
printf("%s %s\n", msg, number_str);
// Free the dynamically allocated memory
OPENSSL_free(number_str);
}

```

- Calculation of **subtraction** and **addition**:

```

// Calculate res = a - b
BN_sub(res, a, b);
// Calculate res = a + b
BN_add(res, a, b);

```

- **Multiplication** calculation (a BN_CTX structure is required in this API):

```

// Calculate res = a * b
BN_mul(res, a, b, ctx)

```

- Calculation **res = a * b mod n** (a BN_CTX structure is required in this API):

```

// Calculate res = a * b mod n
BN_mod_mul(res, a, b, n, ctx)

```

- Calculation **res = a^c mod n** (a BN_CTX structure is required in this API):

```

// Calculate res = a^c mod n
BN_mod_exp(res, a, c, n, ctx)

```

- Calculating **the inverse of an arithmetic remainder**, that is, for an **a** to find **b**, such that it is true:

$a * b \bmod n = 1$, that is $(a * b \bmod n) = 1$.

The value **b** is called the inverse of **a**, in remainder arithmetic **n** and exists only if **gcd(a,b)=1**.

```

// Calculate mod inverse
BN_mod_inverse(b, a, n, ctx);

```

Example

Below is a complete example of using the **bn library**. In it, we initialize three BIGNUM variables, a, b, and n, and then calculate $a^b \bmod n$ and we print **a^b** and **$(a^b \bmod n)$** .

```
/* bn_sample.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM * a)
{
    /* Use BN_bn2hex(a) for hex string
     * Use BN_bn2dec(a) for decimal string */
    char * number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main()
{
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *a = BN_new();
    BIGNUM *b = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *res = BN_new();

    // Initialize a, b, n
    BN_generate_prime_ex(a, NBITS, 1, NULL, NULL, NULL);
    BN_dec2bn(&b, "273489463796838501848592769467194369268");
    BN_rand(n, NBITS, 0, 0);

    //calculate and print res = a*b
```

```
BN_mul(res, a, b, ctx);
printBN("a * b = ", res);

// calculate and print res = a^b mod n
BN_mod_exp(res, a, b, n, ctx);
printBN("a^c mod n = ", res);
return 0;
}
```

Compilation: To compile the program **bn_sample.c** , use the following command.

```
$ gcc bn_sample.c -lcrypto
```

Laboratory Activities

To perform the following activities you will need to use the **SEED Ubuntu 16.04 virtual machine**. In it you will find all the necessary applications (*browser*, *terminal*) as well as tools such as *openssl*, *python*.

Activity 1: Creating a private key

Let p and q be two prime numbers and e be a number chosen appropriately to be relatively prime to $\phi(N)$. The **hexadecimal** values of p , q , and e are given below.

```
p = F7E75FDC469067FFDC4E847C51F452DF
q = E85CED54AF57E53E092113E62F436F4F
e = 0D88C3
```

Write **C code** that calculates and prints the following:

- Modulo N

- The number $\phi(N)$
- The private key d .

In the assignment you submit, you must include, in addition to the **code**, appropriate **screenshots** showing the **execution of the program** in the terminal and the printing of the results.

Note: It should be noted that although the p and q used in this activity are large numbers, they are not large enough to be secure. For the purposes of this exercise and for simplicity, we use 128-bit numbers. In practice, these numbers should be at least 512 bits long.

Activity 2: Encrypting a message

Consider the message M consisting of your first and last name (in Latin characters), in the following format:

Onoma Eponymo

You will need to write the **C code** that encrypts and decrypts this message. The public key (e , N) you will use will be the same as this one

that you used in Activity 1. Furthermore, with the private key **d** that you found in Activity 1, you should verify the result of the encryption, that is, perform decryption.

Hint: To proceed with encryption you will first need to convert your message from an ASCII string to a hexadecimal (hex) string and then convert the hexadecimal string to a BIGNUM using the BN_hex2bn() function. The following python command can be used to convert a simple **ASCII** string to a **hexadecimal** string.

```
$ python -c 'print("My Name".encode("hex"))'  
4d79204e616d65
```

Similarly, to convert a **hexadecimal** string to an **ASCII** string (human readable format) you can use the following python command.

```
$ python -c 'print("4d79204e616d65".decode("hex"))'  
My Name
```

In the assignment you will submit, you should include, in addition to the **code**, appropriate **screenshots** showing the **execution of the program** in the terminal and the printing of the results. You should also show the execution of the python commands for converting strings to/from hexadecimal format.

Activity 3: Decrypting a message

The **public** and **private** keys used in this activity are same as what you calculated in activity 1.

You will first need to write **C code** that decrypts the following C ciphertext, and then (using python) convert it back to an ASCII string in a readable format.

```
C = 7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36
```

In the assignment you will submit, you should include, in addition to the **code**, appropriate **screenshots** showing the **execution of the program** in the terminal and the printing of the results. You should also show the execution of the python commands. for converting strings to/from hexadecimal format.

Activity 4: Signing a message

The public and private keys used in this activity are the same as those in activity 3. Consider a short message **M** of your choice (in Latin characters). You will first need to write **C code** that generates and prints a

digital signature of your message. Then make a small change to the message **M** (such as changing a letter or a number) and sign the modified message again. Finally, compare the two signatures and describe what you observe.

In the assignment you will submit, you should include, in addition to the **code**, appropriate **screenshots** showing the **execution of the program** in the terminal and the printing of the results. You should also show the execution of the python commands. for converting strings to/from hexadecimal format.

Hint (1): To proceed with signing you will first need to convert your message from ASCII string to hexadecimal (hex) using python.

Hint (2): For the purposes of this exercise, we assume that the signature is applied directly to the message and not to the hash value:

Activity 5: Signature Verification

You will first need to write **C code** that performs signature verification in the following two cases:

Case A

Alice receives a message **M** = "Launch a missile." from Bob, along with his signature.

S. We know that Bob's public key is **(e, n)**. Verify that the signature is Bob's or not. The public key and signature (in hexadecimal format) are given below:

M = Launch a missile.


```
S = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F
e = 010001 (this hex value equals to decimal 65537)
n = AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115
```

Next, assume that the signature has been corrupted (corrupted) so that the last byte of the signature changes from 2F to 3F, that is, there is only one bit that has changed. Repeat this activity and describe what will happen during the signature verification process.

Case B

Bob receives the message M ="Please transfer me \$2000.Alice." from Alice, along with the signature of S . We know that Alice's public key is (e, n) . Verify that the signature is Alice's or not. The public key and signature (in hexadecimal format) are given below:

```
M = Please transfer me $2000.Alice.
S = DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320
e = 010001 (this hex value equals to decimal 65537)
n = DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5
```

Tip: you can, if you want, also use the **BN_cmp** function to compare two numbers – see the link:

https://www.openssl.org/docs/man1.0.2/man3/BN_cmp.html

In the assignment you will submit, you should include, in addition to the **code**, appropriate **screenshots** showing the **execution of the program** in the terminal and the printing of the results. You should also show the execution of the python commands. for converting strings to/from hexadecimal format.

Activity 6: Manually verify an X.509 certificate

In this activity, you will need to **verify the digital signature** of a X.509 certificate using the program you developed in the previous activity. An X.509 certificate contains data about a public key and the issuer's signature on the data. You should obtain an actual X.509 certificate from

a **web server** of your choice, extract its issuer's public key, and then use that public key to verify the signature on the certificate.

The steps you need to follow are as follows:

Step 1: Download a certificate from a **real web server of your choice**. You can download _____ certificates using your browser or with openssl from the terminal, using the following command:

```
$ openssl s_client -connect www.example.org:443 -showcerts
```

** In the assignment you submit, **DO NOT use** www.example.org. _____

The result of the command in this example contains (at least) two certificates. The **subject** field (the entry starting with **s:**) of the certificate is `www.example.org`, i.e. this is a certificate for `www.example.org`. The **issuer** field (the entry starting with **i:**) provides the issuer information. The subject field of the second certificate is the same as the issuer field of the first certificate. Basically, the second certificate belongs to an intermediate CA (Certification Authority). _____

```
...
Certificate chain
0 s:/C=US/ST=California/L=Los Angeles/O=Internet Corporation for Assigned Names
and Numbers/CN=www.example.org
i:/C=US/O=DigiCert Inc/CN=DigiCert TLS RSA SHA256 2020 CA1
-----BEGIN CERTIFICATE-----
MIIG1TCCBb2gAwIBAgIQD74IsIVNBXOKsMzhya/uyTANBgkqhkiG9w0BAQsFADBP
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMSkwJwYDVQQDEyBE
...
/sgUKGiQxrjllH/hD4n6p9YJN6FitwAntb7xsV5FKAazVBXmw8isggHOhulr4Xrk
vUzLnF7QYsJhvYtaYrZ2MLxGD+NFI8BkXw==
-----END CERTIFICATE-----

1 s:/C=US/O=DigiCert Inc/CN=DigiCert TLS RSA SHA256 2020 CA1
i:/C=US/O=DigiCert Inc/ OU=www.digicert.com/CN=DigiCert Global Root CA
-----BEGIN CERTIFICATE-----
IIE6jCCA9KgAwIBAgIQCjUI1VwpKwF9+K1lwA/35DANBgkqhkiG9w0BAQsFADBh
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
```

```
...
as6xuwAwapu3r9rxxZf+ingkquqTgLozZXq8oXfp2kUCwA/d5KxTVtzhwoT0Jzl
8ks5T1KESaZMkE4f97Q=
-----END CERTIFICATE-----

2 s:/C=US/O=DigiCert Inc/OU=www.digicert.com/CN=DigiCert Global Root CA
   i:/C=US/O=DigiCert Inc/OU=www.digicert.com/CN=DigiCert Global Root CA
-----BEGIN CERTIFICATE-----
MIIDrzCCAPegAwIBAgIQCDvgVpBCRrGhdWJWZHHHSjANBgkqhkiG9w0BAQUFADBh
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
...
YSEY1QSteDwsOoBrp+uvFRTp2InBuThs4pFsiv9kuXclVzDAGySj4dzp30d8tbQk
CAUw7C29C79Fv1C5qfPrmAESrcilxpg0X40KPMbp1ZWVbd4=
-----END CERTIFICATE-----
```

If using the above command you **only get one certificate**, this means that the certificate you received is signed directly by a **root CA**. The certificate A **root CA** can be downloaded from the Firefox browser installed on the virtual machine. Select Edit —> Preferences —> Privacy —> and then Security —> View Certificates. Search for the issuer name and download its certificate. Copy and paste each of the certificates (the text between

the line containing “Begin CERTIFICATE” and the line containing “End CERTIFICATE”, including these two lines). Name one file **c0.pem** and the other **c1.pem**.

Step 2: Extract the public key (**e, n**) from the issuer certificate. Openssl provides commands to extract some attributes from x509 certificates. We can extract the value of n using the -modulus option.

```
$ openssl x509 -in c1.pem -noout -modulus
```

There is no special command to extract e, but we can print all the fields and easily find the value of e.

```
$ openssl x509 -in c1.pem -text -noout
```

Step 3: Extract the **digital signature** from the server certificate. There is no specific openssl command to extract the signature field. However, we can print all the fields and then copy and paste the

signature block in a file (**Note:** if the signature algorithm used in the certificate is not based on RSA, you will need to find **another certificate**).

```
$ openssl x509 -in c0.pem -text -noout
...
Signature Algorithm: sha256WithRSAEncryption
 84:a8:9a:11:a7:d8:bd:0b:26:7e:52:24:7b:b2:55:9d:ea:30:
 89:51:08:87:6f:a9:ed:10:ea:5b:3e:0b:c7:2d:47:04:4e:dd:
.....
5c:04:55:64:ce:9d:b3:65:fd:f6:8f:5e:99:39:21:15:e2:71: aa:6a:88:82
```

We need to remove the spaces and colons from the data so that we have a hexadecimal string that we can feed into our program. The following commands can achieve this goal. The **tr** command

is a Linux utility for string manipulation. In this case, the **-d** option is used to remove ":" and "blanks" from the data.

```
$ cat signature | tr -d '[:space:]'
84a89a11a7d8bd0b267e52247bb2559dea30895108876fa9ed10ea5b3e0bc7
.....
5c045564ce9db365fdf68f5e99392115e271aa6a8882
```

Step 4: Extract the server certificate body. A Certificate Authority (CA) creates the signature for a server certificate by first calculating the **hash** of the certificate and then signing the hash. To verify the signature, we also need to create the hash of a certificate. Since the hash is created before the signature is calculated, we need to exclude the signature block of a certificate when calculating the hash. Finding the part of the certificate used to create the hash

is quite difficult without a good understanding of the certificate format.

X.509 certificates are encoded according to the **ASN.1** (Abstract Syntax Notation.One) standard, so if we can parse the **ASN.1 structure**, we can easily extract any field from a certificate. Openssl has a command called **asn1parse**, which can be used to parse an X.509 certificate.

```
$ openssl asn1parse -i -in c0.pem
 0:d=0 hl=4 l=1522 cons: SEQUENCE
 4:d=1 hl=4 l=1242 cons: SEQUENCE
```

ÿ

```

8:d=2 hl=2 l= 3 cons: cont[0]
10:d=3 hl=2 l= 1 prim: 13:d=2 hl=2 l= 16          INTEGER          :02
prim: INTEGER
:0E64C5FBC236ADE14B172AEB41C78CB0
...
1236:d=4 hl=2 l= 12 cons: SEQUENCE
1238:d=5 hl=2 l= 3 prim: OBJECT :X509v3 Basic Constraints
1243:d=5 hl=2 l= 1 prim: BOOLEAN :255
1246:d=5 hl=2 l= 2 prim: OCTET STRING [HEX DUMP]:3000
1250:d=1 hl=2 l= 13 cons: SEQUENCE 1252:d=2 hl=2 l= 9          ÿ
prim: OBJECT :sha256WithRSAEncryption
1263:d=2 hl=2 l= 0 prim: NULL 1265:d=1 hl=4 l= 257 prim:
BIT STRING

```

The field starting at ÿ is the certificate body used to create the **hash**. The field starting at ÿ is the signature block. Their offsets *are* the numbers at the beginning of the lines. In our case, the certificate body is from offset 4 to 1249, while the signature block is from 1250 to the end of the file. For X.509 certificates, the starting offset is always the same (i.e. 4), but the end depends on the content length of a certificate. We can use the **-strparse** option to extract

the field from *offset* 4, which will give us the body of the certificate, excluding of the signature block.

```
$ openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
```

Once we have the certificate body, we can calculate its **hash** using the following command:

```
$ sha256sum c0_body.bin
```

(Attention: the above applies if the hash function used for the signature is SHA256. Otherwise, the above command needs to be adjusted).

Step 5: Verify the signature. Now we have all the information, including the **CA public key**, the **CA signature**, and the server **certificate body**. You should run your own program to verify whether the signature is **valid or not**.

Submission

You must submit a detailed **lab report** (in PDF) covering all the actions you took in each activity, your results and your observations. In your description, use screenshots from the terminal, according to the instructions for each activity. You must also include **in the pdf** the **source code** you developed (appropriately commented), in an easily readable format (e.g. screenshot from the editor).